

Rapport de validation du Knowledge Graph NS4KGE

Auteur : Ilyes Tebourski

Date : 20 juillet 2026

Objet : validation systématique, article par article, de l'extraction réalisée par le pipeline NS4KGE, sur les **4 types d'entités** du KG : *datasets*, *metrics*, *KGE models*, *NS methods*.

Corpus : 55 articles scientifiques (négative sampling pour KGE).

1. Objectif

Le KG NS4KGE est peuplé automatiquement par un pipeline à base de LLM. La question posée est : **cette extraction est-elle fiable ?** On mesure, pour chaque type d'entité et chaque article :

- la **précision** : ce que le KG a extrait est-il réellement présent dans l'article ?
- le **recall (relatif)** : ce qui est réellement présent dans l'article a-t-il bien été extrait ?

La validation produit à la fois des **métriques chiffrées** et une **liste localisée et vérifiable** des erreurs (faux positifs) et des oublis probables (candidats faux négatifs), directement exploitable pour **corriger le KG**.

2. Principe méthodologique (commun aux 4 entités)

2.1 Fidélité au pipeline

Le pipeline applique **deux prompts** à deux versions pré-traitées de chaque article :

Prompt	Entrée (fonction du dépôt)	Ce qui est extrait
prompt1_no_results.txt	load_md_no_tables() - article sans les tableaux	proposedNSMethod, mentionedNSMethods, proposedKGModel, mentionedKGModels...
prompt2_tab.txt	load_md_tables_only() - uniquement les tableaux	Configurations[] : Dataset, Metric, KGModel, NSMethod

Conséquence directe et centrale : chaque entité est validée **contre l'entrée que son prompt a réellement vue**.

Un dataset (issu des tableaux) est cherché dans le `tables_only` ; une méthode *mentionnée* (issue de la prose) est cherchée dans le `no_tables`. Les `tables_only/no_tables` sont ****régénérés à la volée** en appelant directement les fonctions du dépôt** `ns4kge-extraction-pipeline`, à partir des `.md` sources de `liste_mds/`.

Il n'y a donc **aucun écart possible** entre ce que voit le validateur et ce que voit le pipeline.

2.2 Validation 100 % déterministe (aucun LLM)

Le KG évalué a été produit par un LLM ; **la validation, elle, est un script Python déterministe** (matching par expressions régulières). Aucun appel API, aucune part d'aléatoire : deux exécutions donnent **strictement le même résultat**. Les scores ne sont donc pas « jugés par une IA », ils résultent d'un comptage mécanique et reproductible.

2.3 Définitions

- **TP** : entité extraite par le KG **et** retrouvée dans sa source.

- **FP** : entité extraite mais **absente** de sa source -> erreur de précision.
- **Candidat faux négatif** : entité présente dans l'article mais **non extraite**.
- **Précision** = $TP / (TP + FP)$.
- **Recall relatif** = $TP / (TP + \text{candidats FN})$. Il est qualifié de **relatif** car les candidats sont cherchés

dans un **vocabulaire global** (l'union des entités extraites sur les 55 articles) : on teste, pour chaque article, si une entité connue ailleurs apparaît chez lui sans avoir été extraite. C'est un proxy robuste mais **indicatif** (certains candidats sont de simples mentions, à valider à la main).

2.4 Distinction de provenance (pour KGE models & NS methods)

Ces deux entités viennent de **deux sources**, ce que l'ontologie distingue explicitement :

- **Évalué** : issu de Configurations (tableaux de résultats) -> le modèle/méthode est **expérimentalement testé**.
- **Mentionné seulement** : issu de la prose (proposed/mentioned) -> **cité** (related work...) sans forcément être évalué.

On calcule donc **4 scores** pour chacune (précision x recall, évalué x mentionné). C'est une valeur propre de l'ontologie : répondre à « *quels modèles sont réellement comparés dans cet article ?* » vs « *lesquels sont juste cités ?* » - impossible sur le texte brut.

3. Résultats - les 12 scores

Entité	Préc. évalué	Préc. mentionné	Recall évalué	Recall mentionné
Datasets	98,9 %	-	97,3 %	-
Metrics	96,5 %	-	96,0 %	-
KGE Models	94,4 %	97,9 %	95,0 %	97,5 %
NS Methods	93,1 %	91,8 %	81,4 %	80,4 %

(Datasets et Metrics ne proviennent que des tableaux -> 2 scores chacun ; KGE Models et NS Methods -> 4 scores chacun.)

Lecture d'ensemble : les scores se dégradent régulièrement de haut en bas, et ce n'est pas un hasard - c'est le reflet direct du degré de standardisation des noms (voir §5).

4. Méthode de matching, par entité

Le principe est le même partout (normalisation + recherche mot-entier tolérante), mais la ****difficulté** du matching augmente** avec le manque de standardisation des noms.

4.1 Datasets - matching verbatim strict

Noms **canoniques** (FB15k-237, WN18RR, UMLS, YAGO3-10). Matching insensible à la casse, séparateurs tolérés, **frontières strictes** pour ne pas confondre un préfixe (FB15k != FB15k-237). Recall restreint aux **cellules** <table> (un dataset doit être une ligne de résultats), avec classement *stats* vs *résultats* par la caption.

4.2 Metrics - matching par alias

Les métriques sont **normalisées par le prompt** (Mean Rank->MR, Hit@10->Hits@10, Acc->Accuracy). On utilise donc des **alias déterministes** par métrique canonique, et on accepte l'en-tête générique Hits@N/Hits@k (le tableau groupe les N) côté précision.

4.3 KGE Models - verbatim + casse + provenance

Noms **standardisés** (TransE, DistMult, ComplEx...). Matching **hybride** : insensible à la casse en précision (ne pas rater ComplEx vs Complex), **sensible à la casse en recall** (les noms sont stylisés comme des mots : Simple!="simple", Rate!="rate"). Exclusion des **non-modèles** (GAN, MLP, BERT, word2vec... ne sont pas des KGEModel : ce sont des frameworks/composants) du vocabulaire de recall.

4.4 NS Methods - le cas le plus difficile (matching en 8 passes)

Voir §5 pour le *pourquoi*. Le matching a nécessité plusieurs couches successives :

1. **Exclusion de Unknown** (placeholder du prompt pour baseline non identifiée - 1634 occurrences).
 2. **Clé canonique** par mots de contenu : dédouble les quasi-doublons (Bernoulli Sampling $\equiv$ Bernoulli Negative Sampling).
 3. **Normalisation des qualificatifs** : NoIGAN (hard) (40%) -> NoIGAN, NSCaching + scratch -> NSCaching, closed world constraints, neg_ratio=1 -> closed world constraints.
 4. **Dérivation d'acronymes** (le levier majeur) : Random Negative Sampling->RNS, Self-Adversarial NS->SANS, avec initiales de tous les mots **et** des seuls mots de contenu.
 5. **Recherche sur l'union des sources** : une méthode *proposée* est écrite Ours dans le tableau mais **nommée en toutes lettres dans la prose** -> il faut chercher dans les deux.
 6. **Dédup par position** : une occurrence de texte (ex. SANS) ne compte qu'**une** fois, même si plusieurs entrées du vocab la matchent par collision d'acronyme.
 7. **Filtrage des adjectifs génériques nus** : Random NS matchait le mot « random » de « Random Walk » -> filtré **en prose** (mais gardé en tableau, où une colonne Random/Uniform est une vraie baseline).
 8. **Exclusion des non-méthodes** génériques (GAN).
-

5. Justification des résultats : pourquoi les scores dégradent (datasets -> NS methods)

C'est le point clé du rapport. **La qualité de la validation suit directement le degré de standardisation des noms d'entités**, qui décroît fortement d'un type à l'autre :

5.1 Datasets (98,9 % / 97,3 %) - noms universels et figés

Les benchmarks KGE sont **ultra-standardisés** : toute la communauté écrit exactement FB15k-237, WN18RR, UMLS. Il n'existe quasiment pas de variante d'écriture. Le matching verbatim suffit -> scores quasi parfaits, **presque aucun pré-traitement nécessaire**. Les 2 seuls FP sont des artefacts d'extraction (Table 1 Dataset, FB13 (FB13_reduced)), pas des variantes ratées.

5.2 Metrics (96,5 % / 96,0 %) - petit ensemble fermé, mais normalisé par le LLM

Il n'existe qu'une **quinzaine** de métriques (MR, MRR, Hits@k, AUC...), donc un ensemble fermé et bien connu.

Seule difficulté : le prompt les **réécrit** (Mean Rank->MR, Hit@10->Hits@10), d'où la couche d'alias.

Fait notable : les 7 FP sont de **vraies erreurs du KG** (métrique Accuracy hallucinée dans 5 articles où le mot n'apparaît nulle part) - la validation les attrape correctement.

5.3 KGE Models (94,4 %-97,9 % / 95,0 %-97,5 %) - noms propres, connus et réutilisés

Les modèles KGE sont des **méthodes célèbres, publiées, réutilisées d'article en article** : TransE, DistMult, ComplEx, RotatE, ConvE... Leur nom est un **nom propre stable**, écrit de la même façon partout (à la casse près). Un article qui utilise TransE écrit TransE. Résultat : matching fiable, scores élevés.

La seule vraie difficulté est la casse stylisée (Complex, Simple) et quelques compounds non splittés (TransE+STC+TCE) que le KG aurait dû séparer - vrais défauts, correctement signalés.

5.4 NS Methods (93,1 %-91,8 % / 81,4 %-80,4 %) - le maillon faible, par nature

C'est ici que tout se complique, et c'est **explicable et attendu** :

- **Les méthodes de negative sampling ne sont PAS standardisées.** Contrairement aux modèles KGE, il n'existe pas de catalogue partagé de noms. Chaque article invente/paraphrase sa propre méthode. Beaucoup n'ont même pas de nom « officiel » : dans plusieurs cas, **le nom de la méthode a dû être construit a posteriori** (par le LLM ou lors de l'annotation), à partir d'une description, car l'article ne lui en donne pas.

-> *Exemple typique : là où un modèle s'appelle simplement TransE partout, une NS method peut être décrite comme « on corrompt les triplets selon la fréquence des entités » sans jamais recevoir de nom court.*

- **Abréviations omniprésentes.** Le KG stocke le nom complet (Self-Adversarial Negative Sampling) alors que l'article n'utilise que l'acronyme (SANS, Self-Adv), ou l'inverse. D'où la nécessité de ****dérivé les acronymes**** - étape inutile pour les datasets/modèles.

- **« Negative Sampling » constamment omis ou abrégé** (Bernoulli Sampling = Bernoulli Negative Sampling), et **qualificatifs expérimentaux collés au nom** (NoIGAN (hard) (40%), TANS w/ Freq) -> d'où les couches de normalisation.

- **Baselines inférées non nommées.** Le LLM attribue souvent une baseline standard (Uniform, Bernoulli, Random, MCMC) à des lignes de tableau **alors que l'article ne la nomme jamais explicitement**. Ce n'est pas nécessairement faux - c'est une inférence plausible - mais c'est ****impossible à prouver par recherche textuelle****. Ces cas pèsent sur la précision *mesurée* (comptés en FP stricts), sans être forcément de vraies erreurs. C'est une **limite de fond** de toute validation par recherche, spécifique aux NS methods.

- **Le recall des mentions (80,4 %) reflète un vrai comportement du KG** : il rate ~1/5 des méthodes citées en related work (CAKE, IGAN, IRGAN, MixGCF, Gibbs NS...). Ce ne sont pas des faux candidats - ce sont de **vrais oublis**, donc un résultat actionnable, pas un artefact de mesure.

En résumé : plus un type d'entité est *nommé de façon standardisée dans la littérature*, plus la validation est facile et le score élevé. Datasets et modèles KGE bénéficient d'une nomenclature communautaire figée ; les NS methods, domaine plus jeune et éclaté, n'ont pas ce consensus - d'où des scores plus bas ****qui traduisent la nature du domaine autant que la qualité de l'extraction****.

6. Analyse des erreurs & décisions honnêtes

- **Précisions données en valeur stricte.** Aucune entité n'a été exclue du dénominateur pour gonfler le score. En particulier, les **baselines inférées** (§5.4) sont comptées comme FP bien qu'elles ne soient pas nécessairement fausses - choix assumé pour ne rien masquer.

- **Corrections de matching légitimes, distinguées du gaming.** Les gains de recall sur les NS methods viennent de corrections de **bugs de comptage** (dédup par position d'une même occurrence ; filtrage d'un mot nu comme « random » qui matchait « Random Walk »), **pas** d'exclusions d'entités. On ne masque aucun vrai oubli : les vraies colonnes de baseline restent comptées.

- **Unknown exclu** des NS methods : c'est un placeholder du prompt, pas une méthode.

• **Non-entités exclues du vocabulaire** (KGE : GAN/MLP/BERT ; NS : GAN) : ce ne sont pas des instances de KGEModel/NSMethod. Leur présence dans le vocab révélait d'ailleurs une **incohérence d'extraction** côté KG (le LLM les a parfois classés comme modèles).

7. Ce que la validation permet

Au-delà des chiffres, la sortie est **exploitable pour corriger le KG** :

- **Faux positifs** -> entrées à corriger/supprimer (ex. Accuracy hallucinée, Table 1 Dataset, compounds non splittés TransE+STC+TCE).
- **Candidats faux négatifs** -> entités probablement à ajouter (baselines de tableaux, méthodes citées en related work non extraites).
- **Distinction évalué / mentionné** validée dans les deux sens (précision et recall), confirmant que le KG capture correctement cette sémantique de l'ontologie.

La validation ne corrige pas automatiquement le KG : elle **localise** les erreurs de façon vérifiable, base d'une correction ciblée (manuelle ou par ré-extraction des articles concernés).

8. Reproductibilité

```
# depuis /home/ilyes/POSTER
python3 validation/validate_datasets.py # datasets -> reports_datasets/
python3 validation/validate_metrics.py # metrics -> reports_metrics/
python3 validation/validate_kgemodels.py # KGE models -> reports_kgemodels/
python3 validation/validate_nsmethods.py # NS methods -> reports_nsmethods/
```

Chaque script lit liste_mds/*.md et ns4kge-kg/per_article/*_merged.json, régénère les tables_only/

no_tables via les fonctions du dépôt, et écrit un rapport détaillé par article + un _SUMMARY.md.

Annexe - chiffres bruts par entité

Entité	Vocab global	Entités vérifiées	FP	Candidats FN
Datasets	73	184	2	5 (résultats) + 1 (stats)
Metrics	18	199	7	8
KGE Models	198	575 (340 éval + 235 ment)	24	17 éval + 6 ment
NS Methods	154 (hors Unknown)	336 (202 éval + 134 ment)	25	43 éval + 30 ment